

TREEMPO: FINE-GRAINED CREDIT ASSIGNMENT FOR LLM TRAINING VIA TOKEN-LEVEL TRAJECTORY BRANCHING

Anonymous authors

Paper under double-blind review

ABSTRACT

We present TreeMPO, a reference-policy-free framework for fine-grained credit assignment in large language model training. Rather than assigning a single scalar reward to an entire reasoning trajectory, TreeMPO constructs a rollout tree, estimates segment-level contribution from mixed-success leaves, and converts these estimates into segment-resolved token-level advantages. The method targets mathematical reasoning while remaining compatible with tool-calling traces that interleave natural-language reasoning, fenced Python calls, tool responses, and final boxed answers. Compared with methods such as TEMPO that exploit shared prefixes among sampled trajectories, TreeMPO actively creates new continuations through guided token-level branching, concentrating exploration at structurally useful and model-plausible decision points. Compared with TreeRPO-style tree credit assignment based on empirical outcome comparisons, TreeMPO uses a probabilistic MAP estimator that estimates signed segment contribution before producing token-level advantages. Experiments use a resource-efficient split pipeline that rolls out from the base policy, generates advantage-weighted training data, fine-tunes LoRA adapters, and validates checkpoint snapshots. Results show consistent but modest validation gains across several model families, stronger gains in some tool-calling-compatible settings, and mixed transfer to broader held-out tests.

1 INTRODUCTION

Sparse end-of-trajectory supervision makes credit assignment a central bottleneck in reinforcement learning for reasoning models. When a long rollout contains many reasoning tokens, optional tool calls, and only a terminal correctness judgment, a single trajectory-level reward provides little guidance about which intermediate decisions should be reinforced or suppressed. The difficulty is especially acute when early errors may be repaired by later reasoning or external tool feedback.

Recent work has begun to exploit shared-prefix and tree structure to densify supervision, including implicit prefix-tree methods, explicit tree-sampling methods, off-policy prefix-aware methods, and fine-grained branching methods (Tran et al., 2025; Yang et al., 2025b; Li et al., 2025; Huang et al., 2025; Wang et al., 2026). These methods show that branching structure exposes local comparisons that are unavailable to flat rollout groups. However, existing approaches often rely on naturally occurring prefix overlap, fixed segment boundaries, offline teacher-generated trees, or heuristic credit estimators that do not explicitly represent uncertainty.

This paper develops fine-grained credit assignment for mathematical reasoning through token-level trajectory branching. The central idea is to turn a small set of full rollouts into a structured tree of alternative continuations, then infer which segments are helpful, harmful, or uncertain from the pattern of correct and incorrect descendants. We call the resulting method TreeMPO (Tree-based Maximum-A-Posteriori Policy Optimization). Figure 1 summarizes the pipeline.

Our implementation is tool-calling compatible: the model receives a question and, in tool-enabled conditions, an optional Python tool specification; it may emit fenced Python markdown blocks, observe tool responses in context, and terminate with a final answer wrapped in `\boxed{}`. Rollouts are organized into trees, branch points are chosen at token granularity, and segment-level posterior

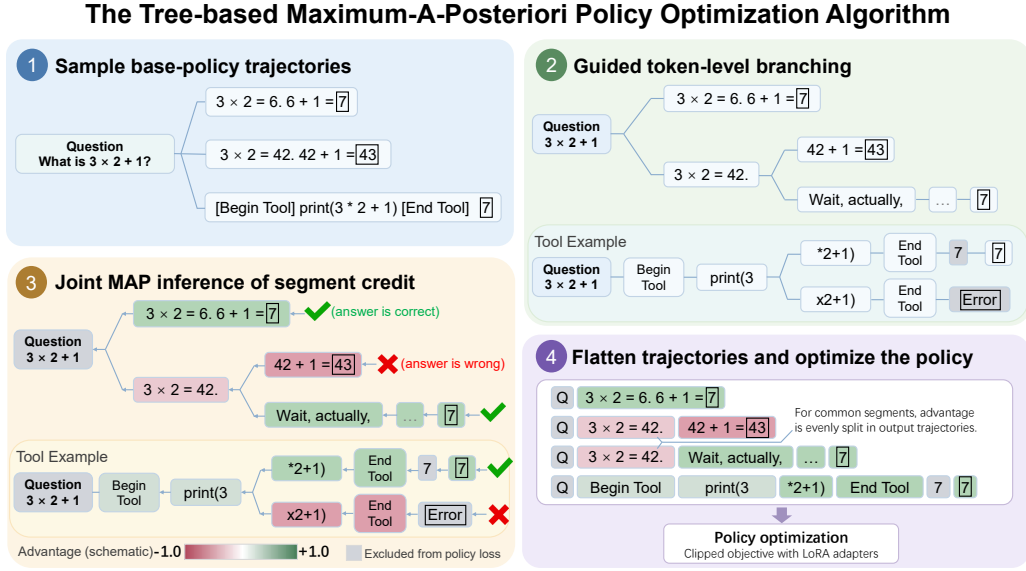


Figure 1: Method overview. TreeMPO samples base-policy trajectories, expands selected token-level branch points, infers segment-level contribution and uncertainty from leaf correctness, and converts the resulting posterior estimates into token-level advantages for adapter training.

estimates are transformed into token-level training advantages. Our experiments use a split one-shot pipeline: collect rollouts from the base model, construct the training set once for each condition, train adapters for multiple epochs, and validate the resulting checkpoints. This isolates the proposed credit-assignment mechanism while keeping the compute budget tractable.

Our contributions are:

- We formulate fine-grained credit assignment as posterior inference over signed segment contributions in a trajectory tree.
- We introduce a guided branching policy that prioritizes model-plausible token-level alternatives while discouraging over-fragmentation and over-concentration, with forced first-token branching used to make the intended alternative explicit.
- We describe a practical training pipeline that converts rollout trees into non-redundant training trajectories with token-level advantages.
- We report split-pipeline LoRA experiments showing small positive validation gains in Qwen2.5-7B and Qwen3-4B settings, together with negative results that identify optimizer sensitivity and mixed held-out transfer.

2 RELATED WORK

Trajectory-level RL for reasoning. Reinforcement learning with verifiable rewards is a standard recipe for mathematical reasoning because final-answer correctness can often be checked automatically. PPO stabilizes policy updates with a clipped surrogate and learned value function (Schulman et al., 2017), while GRPO removes the critic by normalizing outcome rewards across sampled responses (Shao et al., 2024). Both still face the limitation targeted here: terminal feedback can reinforce or suppress many irrelevant tokens along with the decisive reasoning steps.

Process supervision and verifier-based credit. A complementary line densifies supervision by evaluating intermediate reasoning with human step labels, automatically constructed stepwise supervision, explicit first-error localization, implicit process rewards, or alternative process-reward aggregation rules (Lightman et al., 2023; Wang et al., 2024; Hwang et al., 2024; Cui et al., 2025;

Cheng et al., 2025). These methods establish the value of process-level signal, but they introduce process evaluators or explicit error-localization/aggregation mechanisms. TreeMPO instead infers local contribution from correctness variation across deliberately branched policy trajectories.

Tree, segment, and shared-prefix credit. Structured-rollout methods obtain intermediate credit from shared prefixes, flexible segments, or explicit tree search. TEMPO extracts branch-gated corrections from natural prefix trees (Tran et al., 2025); SPO estimates segment-level advantages and includes a tree variant for long traces (Guo et al., 2025); TreeRL branches at high-uncertainty intermediate decisions (Hou et al., 2025); TreeRPO and TreePO use tree structure for local comparisons or subgroup-relative advantages (Yang et al., 2025b; Li et al., 2025); and TreeAdv redistributes leaf advantages over entropy-driven rollout forests (Cao et al., 2026). TRACE allocates rollout budget across prompt roots and intermediate prefixes to increase mixed-reward contrast under a fixed sampling budget (Zou et al., 2026).

TreeMPO is closest to this family, but its differentiating claim is not tree construction alone. It combines model-plausible token-level counterfactual interventions with a joint MAP estimator over latent signed segment contributions and uncertainty, rather than relying on incidental prefix overlap, Monte Carlo segment differences, entropy-only branching, empirical child-group rewards, or direct leaf-advantage redistribution.

Several of these systems report larger gains than ours under their own full training stacks; Table 9 summarizes representative numbers and why they are not directly comparable. We do not include full SPO-tree or TreePO reproductions because their methods are coupled to segment-synchronous rollout schedulers, fallback/replay rules, and objective-specific masking or subgroup normalization, rather than being drop-in replacements for our branching or advantage modules.

Prefix-aware and off-policy tree supervision. Tree-OPO (Huang et al., 2025) uses teacher-generated MCTS prefixes and staged advantage estimation to create prefix-aware off-policy training groups. By contrast, TreeMPO constructs its trees from behavior-policy rollouts and is reference-policy-free: it does not require a reference policy, offline teacher trees, reward models, or process-level supervision. Our split-pipeline experiments reuse these rollouts across optimization epochs, so the training phase is not fully online or strictly on-policy after the adapter changes.

Fine-grained credit without explicit trees. Tree structure is not the only route to critic-free fine-grained credit: GRPO- λ , OAR, and OPPO localize credit using eligibility-style signals, outcome-sensitivity attribution, or Bayesian value recursion within flat or individually scored trajectories (Parthasarathi et al., 2025; Li et al., 2026b;a). TreeMPO instead actively generates local counterfactual continuations and infers shared segment contributions from descendant outcomes.

Branching granularity and long-horizon credit. Long-horizon reasoning creates delayed credit-assignment challenges. GiGPO uses repeated anchor states for local action comparisons (Feng et al., 2025), and APPO argues that influential tool-calling decisions are not confined to coarse workflow stages (Wang et al., 2026). TreeMPO creates local comparisons at token-level reasoning prefixes rather than relying on repeated environment states or procedure-level advantage scaling. Figure 2 contrasts these credit-assignment structures.

Tool-augmented mathematical reasoning and benchmarks. Our tool setting builds on work showing that language models can interleave reasoning with program execution or external actions (Gao et al., 2022; Chen et al., 2022; Yao et al., 2022; Schick et al., 2023). Evaluation uses mathematical reasoning resources including MATH and DeepMath-103K, with additional held-out datasets to test transfer beyond the training distribution (Hendrycks et al., 2021; He et al., 2025).

3 PROBLEM SETTING AND PROTOCOL

We study question-answering models for mathematical reasoning. Each sample provides a problem statement and a ground-truth final answer used only for post hoc correctness judgment. During rollout, the model may produce plain-text reasoning and, when enabled, invoke a Python executor.

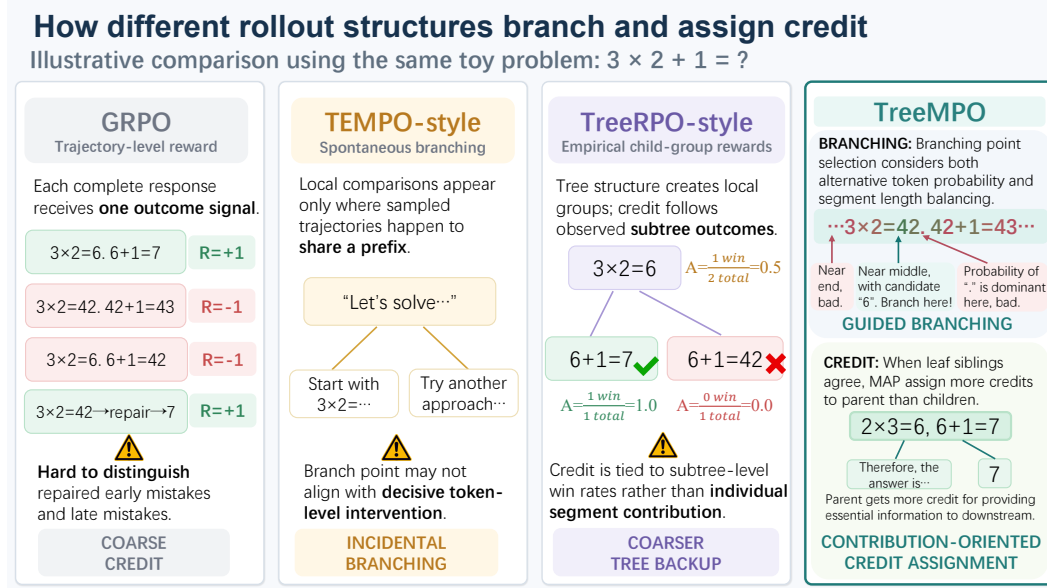


Figure 2: Comparison of credit-assignment structures. Trajectory-level GRPO assigns one normalized outcome signal to every token; TEMPO extracts branch-gated corrections from natural shared prefixes; TreeRPO uses fixed-step tree sampling and empirical child-group rewards; TreeMPO combines guided token-level branching with posterior segment-credit inference.

The main methodological question is how to assign credit inside the resulting mixed text–tool trajectory after only a terminal answer judgment is observed. Our experiments keep the prompting and execution protocol fixed across training-data collection and evaluation so that differences across methods are attributable to branching and credit assignment rather than prompt format changes. Appendix A gives the exact tool-calling and rollout protocol.

4 METHOD

4.1 OVERVIEW

For each question, we first sample a set of complete trunk trajectories. We then expand a rollout tree by branching from selected token positions. Each completed leaf trajectory is judged as correct or incorrect using its extracted final boxed answer. The resulting leaf labels provide supervision for segment-level posterior inference. Posterior estimates are transformed into training advantages for model optimization, while branch selection in the split implementation uses rollout structure and model-side token statistics.

4.2 TRAJECTORY TREE CONSTRUCTION

Branching is token-level rather than step-level. We use *segment* to denote a contiguous generated span between two tree events: the root prompt, a branch point, a tool-response boundary, or a leaf termination. A new branch can be created in two ways:

- **Segment split:** split an existing reasoning segment at an internal token position and regenerate from that point.
- **Node expansion:** branch again from an existing branching node, increasing its branching factor.

Each branch preserves the prefix before the selected token position and then creates a new continuation from that point. This construction makes a tree node correspond to an actual intervention in the model’s decoded trajectory rather than to a manually annotated reasoning step. Appendix A reports

the concrete branch budgets and decoding settings used in experiments. Appendix Table 10 gives implementation-aligned pseudocode for guided branch construction.

4.3 MAP CREDIT ASSIGNMENT MODEL

Let ℓ index judged leaves and let i index non-prompt segments. For each leaf ℓ :

- $y_\ell \in \{+1, -1\}$ is the correctness label.
- $x_{\ell,i} \in \{0, 1\}$ indicates whether segment i lies on the path to leaf ℓ .
- Segment i has mean contribution m_i and log standard deviation u_i .

Leaf-level moments are

$$\mu_\ell = \sum_i x_{\ell,i} m_i, \quad (1)$$

$$\tau_\ell = \sqrt{\sum_i x_{\ell,i} \exp(2u_i) + \varepsilon}, \quad (2)$$

$$z_\ell = y_\ell \mu_\ell / \tau_\ell. \quad (3)$$

We use a probit likelihood,

$$p(y_\ell \mid \{m_i, u_i\}) = \Phi(z_\ell), \quad (4)$$

where Φ is the standard normal CDF.

Our implementation initializes prior means at zero and optimizes the negative log posterior

$$\mathcal{J} = - \sum_\ell \log \Phi(z_\ell) + \frac{1}{2\sigma_m^2} \sum_i m_i^2 + \frac{1}{2\sigma_u^2} \sum_i u_i^2. \quad (5)$$

Optimization is performed with gradient steps and backtracking line search, together with a numerically stable implementation of $\log \Phi$ for large negative margins. Concrete prior scales, iteration counts, and numerical clamps are reported in Appendix A.

4.4 GUIDED TOKEN-LEVEL BRANCHING

The split-pipeline implementation chooses branch points before leaf judgments are available. Branching therefore relies on rollout structure and model-side token statistics rather than posterior uncertainty. For each candidate token position, the branch score multiplies:

1. a structural penalty, and
2. the relative probability of the best alternative token that differs from the already-sampled continuation.

Candidates whose best alternative has relative probability below 0.1 are discarded.

For node expansion, the structural penalty is an exponential branching-factor penalty,

$$\gamma_{\text{node}}(b) = \exp(-k_{\text{node}}(b-1)), \quad (6)$$

with $k_{\text{node}} = 0.35$ in the reported experiments. For segment splitting, the structural penalty increases with the shorter side of the split relative to average trunk length, discouraging very short fragments.

After a branch point and alternative token are selected, the current guided-branching implementation forces that selected token to be the first token of the new branch continuation. This makes the token-level branch explicit rather than merely restarting generation from the shared prefix and hoping that sampling chooses the intended alternative. We keep the non-forced version as an ablation, but use the forced-token version as the default because it better matches the intended token-level intervention.

4.5 FROM POSTERIOR SCORES TO TRAINING ADVANTAGES

After tree construction, each non-root segment receives a signed posterior score $m_i / \exp(u_i)$, where m_i is the inferred mean contribution and $\exp(u_i)$ is the inferred standard deviation. The implementation then applies the same deterministic conversion chain used in the reported runs: divide by a clamped reasoning-token segment length, standardize scores within the tree, set a score to zero if mean subtraction would flip its sign, divide each shared segment score by the number of leaf trajectories that contain it, balance the total positive and negative advantage mass over the generated training set, and finally clip token-level advantages to $[-3, 3]$ before optimization. The generated training samples attach these token-level advantage values aligned with the input sequence. Appendix Table 11 gives pseudocode for the MAP optimization and advantage conversion used by the generation phase.

The downstream training code consumes aligned token sequences, supervised target tokens, attention masks, token-level advantages, and old token log probabilities. The loss is a clipped policy-ratio surrogate over supervised positions,

$$\mathcal{L}_{\text{train}} = -\frac{1}{|\mathcal{S}|} \sum_{t \in \mathcal{S}} \min \left(\rho_t \tilde{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \tilde{A}_t \right), \quad (7)$$

where $\mathcal{S}$ denotes supervised token positions, ρ_t is the ratio between the current token probability and the stored old token probability, and $\tilde{A}_t$ is the token advantage clipped to $[-3, 3]$. Reported non-KL runs use $\epsilon = 0.2$, clamp token log-ratios to $[-20, 20]$, and set $\beta_{\text{KL}} = 0$. This is the implemented optimization interface for injecting segment-level credit into model updates while avoiding the unbounded negative-advantage objective that arises from direct advantage-weighted cross entropy.

4.6 TRAINING-SET CONSTRUCTION

Because many tree leaves share prefixes, naively flattening all leaves would duplicate the same segment-level credit across descendant trajectories. We therefore preserve all informative leaves while normalizing the credit assigned to shared segments.

1. Discard trees whose leaves are uniformly correct or uniformly incorrect, since they provide no contrastive signal for segment-level credit.
2. Compute segment advantages from the chosen credit estimator after leaf judgments are attached.
3. For each segment, count how many completed leaf trajectories contain it and divide that segment’s advantage by this count.
4. Emit each remaining leaf trajectory with token-level advantages inherited from its segments, so shared prefixes contribute a fixed total amount of training signal rather than being multiplied by the number of descendants.
5. Aggregate emitted trajectories across questions under the fixed ordering used by the training pipeline.

Selected trajectories are then emitted under a fixed batching convention so sibling positive and negative samples from the same problem are consumed consecutively; this ordering is an implementation detail rather than an experimental variable.

5 BASELINES AND ABLATIONS

We compare against the base model and a GRPO-style baseline with multiple complete rollouts but no token-level branching. Ablations independently vary exploration and credit estimation: TEMPO-style branching uses spontaneous shared-prefix divergence; TreeRPO-style credit replaces MAP inference with child-group outcome comparisons; TreeRL-style branching uses entropy-based branch selection; and TreeRL-style credit uses a local-global value-difference estimator. This disentangles the source of alternative trajectories from the method used to assign credit.

6 EXPERIMENTAL SETUP

We evaluate TreeMPO and matched baselines on Qwen2.5-7B (Qwen Team, 2024), Qwen3-4B (Yang et al., 2025a), Gemma-3-4B (Gemma Team, 2025), Mistral-7B (Jiang et al., 2023), and Llama-3.1 (Meta Llama Team, 2024), treating tool-enabled and no-tool settings separately. Training and validation use DeepMath, MATH, and NuminaMath (Li et al., 2024); held-out testing adds AMC2023 (math-ai community, 2024), GaoKao Math 2024 (LLMEval Team, 2024), and CollegeMath (Tang et al., 2024). Correctness labels come from a cached OpenRouter judging cascade that accepts exact matches directly, uses inexpensive judges for easy cases, and escalates disagreements to stronger models; Appendix A gives the full protocol. We report pass@1 accuracy as an equal-weight macro average over datasets.

The primary experiments use a split one-shot protocol: collect base-policy rollouts, generate advantage-weighted training data once, train LoRA adapters for multiple epochs, and validate checkpoints. This is cheaper and easier to audit than fully online RL, but mainly tests the quality of the generated credit-assignment data rather than closed-loop policy improvement. A Qwen2.5-7B no-tool semi-online pilot alternates rollout, training-data construction, adapter training, and validation for seven cycles. To reduce compute-budget confounds, GRPO and tree conditions are matched by pre-filter rollout budget, because inference rollout dominates wall-clock and GPU allocation. In the main 8-leaf GRPO versus 16-leaf tree comparison, GRPO receives twice as many raw questions per epoch so both settings generate the same target number of leaf trajectories before all-correct/all-incorrect filtering, shared-segment normalization, and clipping.

7 RESULTS

We report results across all completed experimental conditions. Training rollouts use $T = 0.7$; validation and held-out tests use greedy decoding ($T = 0$) with one trajectory per question. Intervals over “trials” quantify run-to-run variability of the inference and judging pipeline, not policy sampling variance or independently seeded training runs. Each condition is a single training run, so validation-selected checkpoints can be mildly optimistic. Main held-out tests use five rollout trials and all six test datasets; held-out validation uses six trials. Overall, validation gains are modest and repeatable, while broader held-out transfer is mixed. Figure 3 shows the central Qwen2.5 no-tool trend.

7.1 VALIDATION ACCURACY TRENDS

Held-out validation shows that both GRPO and TreeMPO can extract signal from the split pipeline, although gains are modest and non-monotonic. Qwen3-4B has the strongest tool-calling-compatible improvements, while Gemma and Mistral highlight model-family sensitivity. Full validation inventories appear in Appendix B.

7.2 HELD-OUT TESTS

Table 1 reports six-dataset held-out tests using validation-selected checkpoints and five rollout trials. We separate in-distribution and out-of-distribution macro averages because the smaller OOD datasets have higher variance and can obscure transfer patterns. Checkpoint selection and evaluation coverage are in Table 7.

The ID/OOD split sharpens the qualitative picture. In Qwen2.5-7B no-tool, the base model remains strongest on ID datasets and GRPO has the best OOD average. In Qwen2.5-7B with tools, TreeMPO is strongest on ID but GRPO is slightly better OOD, suggesting that segmented credit can improve familiar distributions without always transferring. Qwen3-4B with tools is the clearest positive setting for TreeMPO, while Gemma and Llama show GRPO-favoring ID results but TreeMPO-favoring OOD results; Mistral remains unstable.

7.3 ABLATION STUDY

We separate ablations that are localized within our framework from related systems that require distinct training stacks. Local ablations independently vary branching and credit estimation: TEMPO-

Model and condition	Method	ID avg.	OOD avg.
Qwen2.5-7B no-tool	Base	0.6531$\pm$0.0079	0.6201 $\pm$ 0.0249
Qwen2.5-7B no-tool	GRPO	0.6388 $\pm$ 0.0037	0.6318$\pm$0.0199
Qwen2.5-7B no-tool	TreeMPO	0.6431 $\pm$ 0.0046	0.6177 $\pm$ 0.0164
Qwen2.5-7B no-tool	TreeMPO semi-online	0.6401 $\pm$ 0.0026	0.6242 $\pm$ 0.0199
Qwen2.5-7B + tool	Base	0.6053 $\pm$ 0.0025	0.5431 $\pm$ 0.0296
Qwen2.5-7B + tool	GRPO	0.6125 $\pm$ 0.0061	0.5548$\pm$0.0245
Qwen2.5-7B + tool	TreeMPO	0.6254$\pm$0.0099	0.5458 $\pm$ 0.0193
Qwen3-4B no-tool	Base	0.7067 $\pm$ 0.0073	0.6812 $\pm$ 0.0203
Qwen3-4B no-tool	GRPO	0.7145$\pm$0.0045	0.6853 $\pm$ 0.0079
Qwen3-4B no-tool	TreeMPO	0.7109 $\pm$ 0.0036	0.6884$\pm$0.0162
Qwen3-4B + tool	Base	0.6479 $\pm$ 0.0040	0.6053 $\pm$ 0.0176
Qwen3-4B + tool	GRPO	0.6501 $\pm$ 0.0068	0.6473 $\pm$ 0.0121
Qwen3-4B + tool	TreeMPO	0.6677$\pm$0.0049	0.6711$\pm$0.0160
Gemma-3-4B no-tool	Base	0.5989 $\pm$ 0.0046	0.5649 $\pm$ 0.0269
Gemma-3-4B no-tool	GRPO	0.6113$\pm$0.0049	0.5490 $\pm$ 0.0152
Gemma-3-4B no-tool	TreeMPO	0.6079 $\pm$ 0.0047	0.5734$\pm$0.0194
Mistral-7B no-tool	Base	0.1761 $\pm$ 0.0063	0.1511$\pm$0.0250
Mistral-7B no-tool	GRPO	0.2075$\pm$0.0087	0.1273 $\pm$ 0.0083
Mistral-7B no-tool	TreeMPO	0.1852 $\pm$ 0.0079	0.1282 $\pm$ 0.0218
Llama-3.1 no-tool	Base	0.4081 $\pm$ 0.0060	0.3779 $\pm$ 0.0108
Llama-3.1 no-tool	GRPO	0.4391$\pm$0.0071	0.3838 $\pm$ 0.0136
Llama-3.1 no-tool	TreeMPO	0.4313 $\pm$ 0.0019	0.4020$\pm$0.0160

Table 1: Main held-out test accuracies split by in-distribution and out-of-distribution datasets. Values are equal-weight macro averages with 95% confidence intervals; full per-dataset details are in Appendix D. Bold entries mark the best value in each block.

Qwen2.5-7B no-tool variant	Best val. epoch	Val. base	Best val.	Val. gain	Test epoch	Test macro
GRPO baseline	30	0.6468 $\pm$ 0.0033	0.6538$\pm$0.0021	+0.0069	30	0.6353$\pm$0.0118
TreeMPO, non-forced token	20	0.6399 $\pm$ 0.0026	0.6472 $\pm$ 0.0018	+0.0072	20	0.6294 $\pm$ 0.0085
TreeMPO, forced token	50	0.6436 $\pm$ 0.0025	0.6523 $\pm$ 0.0016	+0.0087	50	0.6304 $\pm$ 0.0081
TEMPO-style branching	70	0.6418 $\pm$ 0.0032	0.6515 $\pm$ 0.0020	+0.0097	70	0.6326 $\pm$ 0.0144
TreeRPO-style credit	40	0.6426 $\pm$ 0.0027	0.6519 $\pm$ 0.0018	+0.0094	40	0.6272 $\pm$ 0.0054
TreeRL-style branching	20	0.6402 $\pm$ 0.0045	0.6480 $\pm$ 0.0041	+0.0078	20	0.6195 $\pm$ 0.0069
TreeRL-style credit	60	0.6474$\pm$0.0044	0.6528 $\pm$ 0.0014	+0.0054	60	0.6334 $\pm$ 0.0062
TreeRL combined	40	0.6449 $\pm$ 0.0032	0.6507 $\pm$ 0.0024	+0.0058	40	0.6341 $\pm$ 0.0110
Branch budget 8	10	0.6446 $\pm$ 0.0040	0.6471 $\pm$ 0.0031	+0.0025	10	0.6274 $\pm$ 0.0076
Branch budget 32	60	0.6428 $\pm$ 0.0026	0.6503 $\pm$ 0.0041	+0.0075	60	0.6333 $\pm$ 0.0105

Table 2: Qwen2.5-7B no-tool ablations. Validation uses six rollout trials; testing uses five trials and six datasets. Gain is best validation minus epoch-0 validation. Bold underlined entries mark column maxima.

style branching uses spontaneous shared-prefix divergence; TreeRPO-style credit uses child-group win-rate comparison; TreeRL-style branching uses entropy-guided branching; and TreeRL-style credit uses local-global value differences. Guided branching uses forced selected branch tokens by default; non-forced branching is retained as a mechanism ablation.

Table 2 shows that most tree-structured variants improve on their own validation baselines, but validation gain does not directly predict broad test accuracy. TEMPO-style branching has the largest validation gain, suggesting that natural prefix diversity already exposes useful alternatives in this no-tool setting. TreeRPO-style credit is close in validation but weaker on the six-dataset test, while TreeRL-style credit and combined variants have comparatively strong held-out macro averages. The forced-token mechanism improves validation over the non-forced version, motivating the default intervention design, although this advantage does not consistently transfer. Closely related systems that require distinct rollout schedulers, objectives, or evaluators are summarized in Appendix Table 9 rather than treated as directly comparable baselines.

7.4 STABILITY DIAGNOSTICS

Negative runs shape our conclusions: the method is sensitive to optimizer aggressiveness, adapter configuration, and model family. Appendix C summarizes unstable settings, and Appendix F shows

that increasing branch budget from 8 to 32 slightly improves held-out test macro accuracy but not monotonically enough to be a primary claim.

7.5 TOOL-COMPATIBLE CASE STUDY

A representative Qwen2.5-7B tool-enabled TreeMPO trajectory illustrates why reference-policy-free training with tools remains sensitive to base-model tool competence. The model emits balanced Python fences but continues with a placeholder-like execution block instead of cleanly yielding control for a tool response; later it repairs the answer manually. The segment advantages 0.000, -0.870 , $+0.434$ assign negative credit to the confused tool-state transition and positive credit to the recovery. The full colored trajectory and prompt are in Appendix I.

8 DISCUSSION

The results point to a trade-off among credit granularity, rollout budget, and optimization stability. GRPO is inexpensive and stable because it assigns one group-normalized terminal signal to full responses, but this can blur late repairs and intermediate mistakes. Tree-based methods spend more inference budget to expose local alternatives and train on variable segment-level advantages; this helps when trees contain meaningful correctness contrasts, but can increase update variance. This connects to observations that saturated reasoning data can be “too correct to learn” (Liang et al., 2026): only mixed-correctness trees provide informative contrast, and Appendix Table 3 shows that only 36–53% of TreeMPO training-rollout questions have this property.

Tool-calling compatibility appears to make segmented credit more valuable in some settings because tool-enabled trajectories can contain error–feedback–repair transitions, where early reasoning is harmful but later tool-informed reasoning is helpful. This is consistent with the case study and the strongest Qwen3 tool result favoring TreeMPO. The ablations should therefore be read as a method-characteristic map rather than a global ranking: spontaneous or entropy-based branching can work when models naturally diversify, win-rate and local–global backups can work when correctness changes align with subtree boundaries, and the MAP estimator is better matched to cases where shared segments jointly explain mixed descendants. Forced-token branching is the default because the branch score evaluates a specific alternative token, so forcing that token makes the generated intervention match the scored decision.

9 LIMITATIONS AND FUTURE WORK

TreeMPO incurs additional rollout structure per accepted learning signal and is evaluated primarily in a split one-shot rather than fully online regime. Gains are modest and transfer unevenly, motivating larger-scale online experiments, stronger optimizer controls, and repeated independent seeds. The MAP estimator assumes additive segment contributions and relies on final-answer extraction. Our ablations approximate several related tree-credit ideas within one infrastructure, but full SPO-tree or TreePO reproductions would require separate segment-synchronous rollout, replay, masking, or subgroup-normalized training pipelines (Guo et al., 2025; Li et al., 2025).

10 CONCLUSION

We presented TreeMPO, a trajectory-tree method that branches reasoning trajectories at token granularity, infers segment-level contribution from mixed-success descendants, and converts those estimates into training signals. Split-pipeline experiments show that TreeMPO can outperform baselines under some configurations, especially when recoverable intermediate mistakes appear, but gains are modest and transfer is uneven. We view TreeMPO as a useful option when training scale and rollout budget can support finer credit assignment, not as a uniformly dominant replacement for GRPO.

ETHICS STATEMENT

This work studies mathematical reasoning benchmarks and does not involve human-subject data collection. The main ethical risks are ordinary risks of improving automated reasoning systems,

including possible misuse and over-reliance on generated reasoning traces. We mitigate these risks by evaluating only answer correctness on public mathematical datasets and by reporting negative and mixed results alongside positive findings.

REPRODUCIBILITY STATEMENT

The paper reports the model families, rollout protocol, training hyperparameters, validation cadence, and testing protocol used in the experiments. The implementation keeps explicit judging/scoring passes and concise run summaries for validation and testing so that experiment outputs can be audited and regenerated.

AI USE STATEMENT

Generative AI tools were used to assist with code implementation, experiment orchestration, log inspection, result aggregation, and manuscript editing. All reported results, claims, code changes, and paper text were reviewed by the authors, who take responsibility for the final content of this work.

REFERENCES

- Lang Cao, Hui Ruan, Yongqian Li, Peng Chao, Wu Ning, Haonan Song, Renhong Chen, and Yitong Li. Tree-structured advantage redistribution for group-based RL, 2026. *arXiv:2601.03703*.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022. URL <https://arxiv.org/abs/2211.12588>.
- Jie Cheng, Gang Xiong, Ruixi Qiao, Lijun Li, Chao Guo, Junle Wang, Yisheng Lv, and Fei-Yue Wang. Stop summation: Min-form credit assignment is all process reward model needs for reasoning, 2025. *NeurIPS 2025*.
- Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, Jiarui Yuan, Huayu Chen, Kaiyan Zhang, Xingtai Lv, Shuo Wang, Yuan Yao, Xu Han, Hao Peng, Yu Cheng, Zhiyuan Liu, Maosong Sun, Bowen Zhou, and Ning Ding. Process reinforcement through implicit rewards, 2025. *arXiv:2502.01456*.
- Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for LLM agent training, 2025. *NeurIPS 2025*; *arXiv:2505.10978*.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. *arXiv preprint arXiv:2211.10435*, 2022. URL <https://arxiv.org/abs/2211.10435>.
- Gemma Team. Gemma 3 technical report, 2025. *arXiv:2503.19786*.
- Yiran Guo, Lijie Xu, Jie Liu, Dan Ye, and Shuang Qiu. Segment policy optimization: Effective segment-level credit assignment in RL for large language models, 2025. *NeurIPS 2025*; *arXiv:2505.23564*.
- Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Deepmath-103k: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning. *arXiv preprint arXiv:2504.11456*, 2025. URL <https://arxiv.org/abs/2504.11456>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *Advances in Neural Information Processing Systems*, 2021. URL <https://arxiv.org/abs/2103.03874>.

- Zhenyu Hou, Ziniu Hu, Yujiang Li, Rui Lu, Jie Tang, and Yuxiao Dong. TreeRL: LLM reinforcement learning with on-policy tree search. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.
- Bingning Huang, Tu Nguyen, and Matthieu Zimmer. Tree-OPO: Off-policy monte carlo tree-guided advantage optimization for multistep reasoning, 2025. arXiv:2509.09284.
- Hyeonbin Hwang, Doyoung Kim, Seungone Kim, Seonghyeon Ye, and Minjoon Seo. Self-Explore: Enhancing mathematical reasoning in language models with fine-grained rewards. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mistral 7B, 2023. arXiv:2310.06825.
- Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. NuminaMath. https://github.com/project-numina/aimo-progress-prize/blob/main/report/numina_dataset.pdf, 2024.
- Yizhi Li, Qingshui Gu, Zhoufutu Wen, Ziniu Li, Tianshun Xing, Shuyue Guo, Tianyu Zheng, Xin Zhou, Xingwei Qu, Wangchunshu Zhou, Zheng Zhang, Wei Shen, Qian Liu, Chenghua Lin, Jian Yang, Ge Zhang, and Wenhao Huang. TreePO: Bridging the gap of policy optimization and efficacy and inference efficiency with heuristic tree-based modeling, 2025. arXiv:2508.17445.
- Yu Li, Rui Miao, Tian Lan, and Zhengling Qi. OPPO: Bayesian value recursion for token-level credit assignment in LLM reasoning, 2026a. arXiv:2605.21851.
- Ziheng Li, Liu Kang, Feng Xiao, Luxi Xing, Qingyi Si, Zhuoran Li, Weikang Gong, Deqing Yang, Yanghua Xiao, and Hongcheng Guo. Outcome-grounded advantage reshaping for fine-grained credit assignment in mathematical reasoning, 2026b. arXiv:2601.07408.
- Zhenwen Liang, Yujun Zhou, Sidi Lu, Xiangliang Zhang, Haitao Mi, and Dong Yu. Too correct to learn: Reinforcement learning on saturated reasoning data, 2026. arXiv:2604.18493.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023. URL <https://arxiv.org/abs/2305.20050>.
- LLMEval Team. LLMEval-Gaokao2024-Math. <https://github.com/llmeval/llmeval-Gaokao2024-Math>, 2024.
- math-ai community. amc23: A mathematical reasoning benchmark from the 2023 american mathematics competition. <https://huggingface.co/datasets/math-ai/amc23>, 2024.
- Meta Llama Team. The Llama 3 herd of models, 2024. arXiv:2407.21783.
- Prasanna Parthasarathi, Mathieu Reymond, Boxing Chen, Yufei Cui, and Sarath Chandar. GRPO-  : Credit assignment improves LLM reasoning, 2025. arXiv:2510.00194.
- Qwen Team. Qwen2.5 technical report, 2024. arXiv:2412.15115.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dess  , Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023. URL <https://arxiv.org/abs/2302.04761>.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization algorithms, 2017. arXiv:1707.06347.

- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Zhengyang Tang, Xingxing Zhang, Benyou Wang, and Furu Wei. MathScale: Scaling instruction tuning for mathematical reasoning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 47885–47900. PMLR, 2024.
- Hieu Tran, Zonghai Yao, and Hong Yu. Exploiting tree structure for credit assignment in RL training of LLMs, 2025. arXiv:2509.18314.
- Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- Xucong Wang, Ziyu Ma, Yong Wang, Yuxiang Ji, Shidong Yang, Guanhua Chen, Pengkun Wang, and Xiangxiang Chu. APPO: Agentic procedural policy optimization, 2026. arXiv:2606.12384.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chuji Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025a. arXiv:2505.09388.
- Zhicheng Yang, Zhijiang Guo, Yinya Huang, Xiaodan Liang, Yiwei Wang, and Jing Tang. TreeRPO: Tree relative policy optimization, 2025b. arXiv:2506.05183.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022. URL <https://arxiv.org/abs/2210.03629>.
- Zhe Yuan, Yipeng Zhou, Jinghan Li, Xinyuan Chen, Bowen Deng, Zhiqian Chen, and Liang Zhao. Lampo: A lambda style policy optimization for reasoning language models, 2026. arXiv:2605.21235.
- Heming Zou, Qi Wang, Yun Qu, Yuhang Jiang, Lizhou Cai, Yixiu Mao, Ru Peng, Xin Xu, Weijie Liu, Kai Yang, Saiyong Yang, and Xiangyang Ji. TRACE: A unified rollout budget allocation framework for efficient agentic reinforcement learning, 2026. EMNLP 2026 Main Conference; arXiv:2606.11119.

A IMPLEMENTATION AND PROTOCOL DETAILS

Tool-calling protocol. Each prompt contains the problem statement and, when tool calls are enabled, a specification that Python can be called by emitting a fenced markdown block. The model may interleave reasoning and tool calls. When the exact closing-fence control pattern is detected, generation pauses, the tool executes, and the tool response is appended to the context. The model is instructed to present its final answer in `\boxed{}`. If generation reaches a control handoff without a boxed answer, the framework requests continuation until a boxed answer is produced or rollout termination conditions are hit.

Correctness judging. Training-rollout labels, held-out validation labels, and held-out test labels are produced by a separate judging-and-scoring pass. The judging input consists of the original question, the reference answer, and the model’s extracted final answer; the judging prompt asks

the judge to compare only the model answer against the reference answer and to return a first-line verdict of `\boxed{correct}` or `\boxed{incorrect}`. Exact string matches between the extracted model answer and the reference answer are accepted without an API call. Empty extracted answers are marked incorrect.

For non-exact matches, we use a three-stage OpenRouter cascade to balance accuracy and cost efficiency.

Stage 1. We query three lower-cost judges independently:

```
deepseek/deepseek-v4-flash
openai/gpt-5-nano
google/gemini-2.5-flash-lite
```

If all three agree, their unanimous verdict is used.

Stage 2. If Stage 1 disagrees, we query two stronger judges:

```
deepseek/deepseek-v4-pro
openai/gpt-4.1-mini
```

If these two models agree, that verdict is used.

Stage 3. Remaining disagreements are escalated to the final judge:

```
openai/gpt-5-mini
```

Its verdict is final. The first attempt for each judge uses temperature 0; retry attempts use temperature 0.7. Invalid judge outputs and transient API failures are retried up to three times for that judge. Fatal API errors or repeated invalid outputs fail the judging job rather than silently caching an incorrect verdict.

Judgments are cached by split, question identifier, extracted model answer string, and cache version in readable JSONL cache chunks of 1,000 question identifiers. The judging job acquires a per-cache-chunk lock with a heartbeat to avoid concurrent writes to the same chunk, records cache-hit and cache-miss statistics, and appends Stage-3 escalation cases to a global audit log. To reduce storage while preserving auditability, unanimous Stage-1 cache records store only the final decision; cases requiring Stage 2 or Stage 3 keep the raw judge outputs used to resolve the disagreement.

Rollout and tree hyperparameters. In the default configuration for the main tool-using setup, each tree begins with 4 trunk trajectories and expands to 16 total trajectories. Branch-budget ablations use 8 and 32 total trajectories. We keep a fixed decoding temperature within each tree; in the reported configurations, this value is $T = 0.7$ for trunk generation, continuation, and branch expansion.

Rollout-budget balancing. For the main chunked comparison, GRPO uses 8 complete leaf trajectories per question, while the tree setting uses 16 leaf trajectories per question. Because rollout inference is the bottleneck, we approximately equalize rollout work per training epoch by assigning GRPO twice as many raw questions per chunk: 250 questions for GRPO and 125 questions for tree rollouts. Thus both settings target 2,000 generated leaf trajectories per epoch before correctness filtering. Direct run checks confirm this layout for the Qwen2.5 no-tool run: the first GRPO chunk contains question indices 0–249 with 8 leaves per question, and the first tree chunk contains question indices 0–124 with 16 leaves per question, giving 2,000 leaves in both chunks. The first-run timing summaries are also comparable for Qwen2.5: 235 seconds per GRPO no-tool chunk versus 207 seconds per tree no-tool chunk, and 237 seconds per GRPO tool chunk versus 210 seconds per tree tool chunk. After judging and filtering, the number of accepted training trajectories can differ because all-correct and all-incorrect questions are removed. This is expected rather than a failure of the budget model: those removed samples carry no contrastive credit signal, and the scarce resource being controlled is the cost of generating candidate trajectories, not the final number of supervised token updates.

Table 3 shows that rollout efficiency varies substantially by model family and tool setting. For Qwen2.5-7B and Qwen3-4B, enabling tools reduces the all-correct fraction and increases the mixed-

Model	Setting	Questions	All correct	All incorrect	Mixed	Filtered	Train traj.	Traj./question
Qwen2.5-7B	No-tool	8,750	3,883 (44.4%)	1,431 (16.4%)	3,436 (39.3%)	60.7%	54,976	6.3
Qwen2.5-7B	Tool	8,750	3,228 (36.9%)	1,496 (17.1%)	4,026 (46.0%)	54.0%	64,416	7.4
Qwen3-4B	No-tool	6,250	3,120 (49.9%)	847 (13.6%)	2,283 (36.5%)	63.5%	36,528	5.8
Qwen3-4B	Tool	6,250	2,637 (42.2%)	934 (14.9%)	2,679 (42.9%)	57.1%	42,864	6.9
Gemma-3-4B	No-tool	6,250	2,421 (38.7%)	1,526 (24.4%)	2,303 (36.8%)	63.2%	36,848	5.9
Mistral-7B	No-tool	6,250	149 (2.4%)	3,112 (49.8%)	2,989 (47.8%)	52.2%	47,824	7.7
Llama-3.1	No-tool	6,250	1,112 (17.8%)	1,843 (29.5%)	3,295 (52.7%)	47.3%	52,720	8.4

Table 3: Training-rollout correctness breakdown for TreeMPO generation runs. “Filtered” is the share of questions with all-correct or all-incorrect leaves, which produce no within-question contrastive credit signal and are removed before training trajectory construction. “Train traj.” counts accepted leaf trajectories after this question-level filtering.

Method	Questions	Train traj.	Mean $ \bar{A} $	Median $ \bar{A} $
GRPO	17,500	53,200	0.825	0.577
TreeMPO	8,750	54,976	0.355	0.281

Table 4: Training-set advantage-scale audit for Qwen2.5-7B no-tool runs. $|\bar{A}|$ is the absolute average token advantage for an emitted leaf trajectory, as recorded by the generation statistics after shared-segment normalization and token clipping but before final positive/negative mass balancing. The lower mean for TreeMPO reflects shared-segment advantage normalization across descendant leaves rather than weaker rollout correctness alone.

correctness fraction by roughly 6–7 points, consistent with tool feedback creating more recoverable error–repair variation. Across no-tool models, stronger or more saturated models such as Qwen3-4B produce many all-correct trees, while weaker models such as Mistral produce many all-incorrect trees. Llama has the highest mixed-correctness rate in this table, indicating many informative rollouts, but the main results show that informative contrast alone is insufficient without stable optimization and good transfer.

MAP-credit optimization details. The hyperparameter configuration uses $\sigma_m = 1.0$, $\sigma_u = 1.0$, $\varepsilon = 10^{-6}$, 120 optimization iterations, and log-standard-deviation clamps in $[-4, 2]$. These settings are fixed across the main TreeMPO comparisons unless an ablation explicitly changes the credit-estimation rule.

Datasets. Training and validation are built from DeepMath, MATH, and NuminaMath (Li et al., 2024). The source pools contain larger dataset-specific slices, and each experiment consumes deterministic prefixes or chunks from those pools after reserving disjoint validation ranges. The validation split contains 1,000 validation samples per dataset. The held-out test split aggregates DeepMath, MATH, NuminaMath, AMC2023 (math-ai community, 2024), GaoKao Math 2024 (LLMEval Team, 2024), and CollegeMath (Tang et al., 2024). For datasets with more than 1,000 test rows, 1,000 are sampled with seed 42; for DeepMath, which lacks a dedicated test split in the evaluation pipeline, evaluation rows are drawn from the training split after clearing the reserved training and validation examples.

Training and evaluation protocol. The main stable runs use chunked one-shot training, where each epoch consumes one generated training chunk and recovery restarts at whole-chunk boundaries. Primary experiments use LoRA rank 32 and learning rate 10^{-6} , with Adam state and learning-rate warmup enabled unless stated otherwise. Held-out validation is run at epoch 0 and then every 10 epochs for long runs, with multiple validation rollout trials used to reduce sampling noise. We use LoRA-based fine-tuning for the primary experiments; full-model training is deferred because queuing and memory costs are substantially higher.

B FULL VALIDATION INVENTORY

Table 5 gives the full held-out validation inventory used for checkpoint selection and trend analysis.

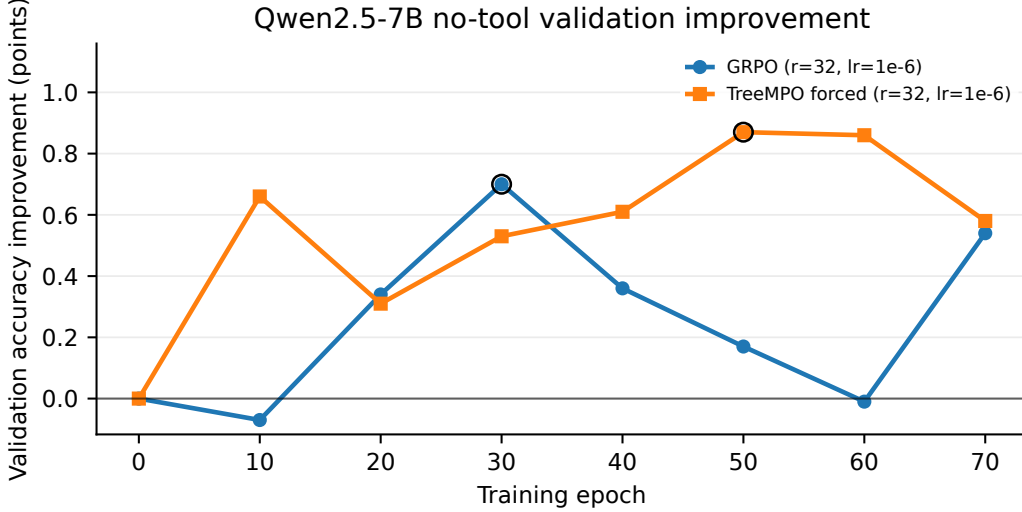


Figure 3: Validation accuracy improvement over the base checkpoint for Qwen2.5-7B in the no-tool setting. Both lines use LoRA rank 32 and learning rate 10^{-6} ; the y-axis reports percentage-point change relative to each method’s epoch-0 validation accuracy.

Model and condition	Method	Trials	Epochs audited	Base	Best epoch	Best	Gain	Last
Qwen2.5-7B no-tool	GRPO	6	0,10,...,70	0.6468 $\pm$ 0.0033	30	0.6538 $\pm$ 0.0021	+0.0069	0.6522 $\pm$ 0.0039
Qwen2.5-7B no-tool	TreeMPO	6	0,10,...,70	0.6436 $\pm$ 0.0025	50	0.6523 $\pm$ 0.0016	+0.0087	0.6494 $\pm$ 0.0035
Qwen2.5-7B + tool	GRPO	6	0,10,...,70	0.6142 $\pm$ 0.0033	30	0.6246 $\pm$ 0.0024	+0.0104	0.6171 $\pm$ 0.0046
Qwen2.5-7B + tool	TreeMPO	6	0,10,...,70	0.6134 $\pm$ 0.0034	50	0.6236 $\pm$ 0.0033	+0.0101	0.6181 $\pm$ 0.0016
Qwen3-4B no-tool	GRPO	6	0,10,...,50	0.6901 $\pm$ 0.0021	50	0.6991 $\pm$ 0.0038	+0.0089	0.6991 $\pm$ 0.0038
Qwen3-4B no-tool	TreeMPO	6	0,10,...,50	0.6894 $\pm$ 0.0017	30	0.6944 $\pm$ 0.0024	+0.0050	0.6849 $\pm$ 0.0047
Qwen3-4B + tool	GRPO	6	0,10,...,50	0.6446 $\pm$ 0.0030	40	0.6617 $\pm$ 0.0046	+0.0172	0.6521 $\pm$ 0.0044
Qwen3-4B + tool	TreeMPO	6	0,10,...,50	0.6434 $\pm$ 0.0034	50	0.6605 $\pm$ 0.0023	+0.0171	0.6605 $\pm$ 0.0023
Gemma-3-4B no-tool	GRPO	6	0,10,...,50	0.5640 $\pm$ 0.0022	50	0.5647 $\pm$ 0.0032	+0.0007	0.5647 $\pm$ 0.0032
Gemma-3-4B no-tool	TreeMPO	6	0,10,...,50	0.5621 $\pm$ 0.0030	40	0.5643 $\pm$ 0.0029	+0.0023	0.5594 $\pm$ 0.0034
Mistral-7B no-tool	GRPO	6	0,10,...,50	0.1747 $\pm$ 0.0043	40	0.1802 $\pm$ 0.0013	+0.0055	0.1729 $\pm$ 0.0021
Mistral-7B no-tool	TreeMPO	6	0,10,...,50	0.1747 $\pm$ 0.0019	10	0.1819 $\pm$ 0.0031	+0.0072	0.1788 $\pm$ 0.0023
Llama-3.1 no-tool	GRPO	6	0,10,...,50	0.4415 $\pm$ 0.0038	30	0.4684 $\pm$ 0.0018	+0.0269	0.4609 $\pm$ 0.0064
Llama-3.1 no-tool	TreeMPO	6	0,10,...,50	0.4454 $\pm$ 0.0035	40	0.4589 $\pm$ 0.0029	+0.0135	0.4568 $\pm$ 0.0053

Table 5: Audited held-out validation results. Values are pass@1 macro averages on the validation mixture, reported as mean with 95% confidence interval over rollout trials. All rows use six validation trials and epochs that are 0 or multiples of 10. Bold underlined entries mark the largest value within each model-condition comparison block.

C STABILITY AND NEGATIVE-RUN DIAGNOSTICS

Several failed or unhelpful runs influenced the final experiment design. Qwen2.5-7B TreeMPO at LoRA rank 32 with learning rate 5×10^{-5} collapsed in earlier validation, while the same rank with learning rate 10^{-6} remained stable. SGD/no-warmup variants did not show useful trends and were removed from the main schedule. Mistral remains difficult under this recipe: its absolute accuracy is low, and TreeMPO is below the base model on held-out testing despite a small validation gain. These outcomes motivate treating adapter rank, learning rate, optimizer state, and model-family sensitivity as substantive empirical factors rather than incidental implementation details.

D FULL HELD-OUT TEST DETAILS

Table 6 reports the per-dataset held-out test results that underlie the compact main-text table.

Model and condition	Method	In-distribution datasets			Out-of-distribution datasets			Group averages	
		DeepMath	MATH	NuminaMath	AMC 2023	GaoKao 2024	CollegeMath	ID avg.	OOD avg.
Qwen2.5-7B no-tool	Base	0.6034±0.0030	0.7400±0.0056	0.6160±0.0211	0.5400±0.0480	0.6000±0.0511	0.7204±0.0029	0.6531±0.0079	0.6201±0.0249
Qwen2.5-7B no-tool	GRPO	0.5886±0.0028	0.7338±0.0056	0.5940±0.0133	0.5200±0.0475	0.6692±0.0185	0.7062±0.0042	0.6388±0.0037	0.6318±0.0199
Qwen2.5-7B no-tool	TreeMPO	0.6050±0.0059	0.7362±0.0031	0.5880±0.0073	0.5900±0.0480	0.5615±0.0185	0.7016±0.0043	0.6431±0.0046	0.6177±0.0164
Qwen2.5-7B no-tool	TreeMPO semi-online	0.6078±0.0011	0.7506±0.0031	0.5620±0.0039	0.5300±0.0475	0.6231±0.0048	0.7194±0.0037	0.6401±0.0026	0.6242±0.0199
Qwen2.5-7B + tool	Base	0.5806±0.0042	0.6812±0.0042	0.5540±0.0048	0.4650±0.0720	0.5154±0.0511	0.6488±0.0082	0.6053±0.0025	0.5431±0.0296
Qwen2.5-7B + tool	GRPO	0.5808±0.0065	0.6986±0.0053	0.5580±0.0114	0.4750±0.0410	0.5385±0.0413	0.6508±0.0029	0.6125±0.0061	0.5548±0.0245
Qwen2.5-7B + tool	TreeMPO	0.5846±0.0062	0.7036±0.0046	0.5880±0.0243	0.4600±0.0504	0.5231±0.0185	0.6542±0.0039	0.6254±0.0099	0.5458±0.0193
Qwen3-4B no-tool	Base	0.6716±0.0035	0.8244±0.0032	0.6240±0.0202	0.5800±0.0392	0.7615±0.0282	0.7022±0.0022	0.7067±0.0073	0.6812±0.0203
Qwen3-4B no-tool	GRPO	0.6846±0.0038	0.8250±0.0069	0.6340±0.0078	0.6250±0.0219	0.7231±0.0369	0.7078±0.0042	0.7145±0.0045	0.6853±0.0079
Qwen3-4B no-tool	TreeMPO	0.6840±0.0092	0.8368±0.0047	0.6120±0.0073	0.5850±0.0250	0.7769±0.0554	0.7034±0.0022	0.7109±0.0036	0.6884±0.0162
Qwen3-4B + tool	Base	0.6078±0.0056	0.7458±0.0044	0.5900±0.0164	0.5850±0.0250	0.5692±0.0440	0.6616±0.0029	0.6479±0.0040	0.6053±0.0176
Qwen3-4B + tool	GRPO	0.6288±0.0063	0.7734±0.0073	0.5480±0.0144	0.6650±0.0250	0.6077±0.0369	0.6692±0.0069	0.6501±0.0068	0.6473±0.0121
Qwen3-4B + tool	TreeMPO	0.6340±0.0095	0.7670±0.0021	0.6020±0.0096	0.6500±0.0410	0.6846±0.0282	0.6788±0.0035	0.6677±0.0049	0.6711±0.0160
Gemma-3-4B no-tool	Base	0.4780±0.0067	0.7606±0.0043	0.5580±0.0114	0.4800±0.0646	0.5692±0.0282	0.6456±0.0049	0.5989±0.0046	0.5649±0.0269
Gemma-3-4B no-tool	GRPO	0.4810±0.0059	0.7610±0.0027	0.5920±0.0114	0.4300±0.0240	0.5769±0.0238	0.6402±0.0039	0.6113±0.0049	0.5490±0.0152
Gemma-3-4B no-tool	TreeMPO	0.4846±0.0122	0.7670±0.0078	0.5720±0.0130	0.5050±0.0567	0.5769±0.0238	0.6382±0.0047	0.6079±0.0047	0.5734±0.0194
Mistral-7B no-tool	Base	0.1910±0.0092	0.1474±0.0035	0.1900±0.0124	0.0650±0.0332	0.2154±0.0452	0.1728±0.0040	0.1761±0.0063	0.1511±0.0250
Mistral-7B no-tool	GRPO	0.2136±0.0062	0.1528±0.0063	0.2560±0.0237	0.0500±0.0155	0.1538±0.0238	0.1782±0.0069	0.2075±0.0087	0.1273±0.0083
Mistral-7B no-tool	TreeMPO	0.2014±0.0058	0.1562±0.0061	0.1980±0.0180	0.0350±0.0250	0.1769±0.0511	0.1728±0.0032	0.1852±0.0079	0.1282±0.0218
Llama-3.1 no-tool	Base	0.3250±0.0065	0.4954±0.0032	0.4040±0.0100	0.2850±0.0332	0.3846±0.0238	0.4642±0.0052	0.4081±0.0060	0.3779±0.0108
Llama-3.1 no-tool	GRPO	0.3666±0.0111	0.5168±0.0071	0.4340±0.0159	0.2750±0.0268	0.4077±0.0452	0.4688±0.0051	0.4391±0.0071	0.3838±0.0136
Llama-3.1 no-tool	TreeMPO	0.3314±0.0045	0.5146±0.0118	0.4480±0.0114	0.3200±0.0098	0.4077±0.0612	0.4784±0.0080	0.4313±0.0019	0.4020±0.0160

Table 6: Held-out test results by dataset and dataset group. Values are pass@1 accuracies with 95% confidence intervals from per-trial scores. ID and OOD averages are equal-weight macro averages over the three datasets in each group. Bold underlined entries mark the largest value within each model-condition comparison block. Table 8 separately compares forced and non-forced TreeMPO variants.

Model and condition	Method	Selected epoch	Testing trials	Dataset coverage
Qwen2.5-7B no-tool	Base	0	5	6/6
Qwen2.5-7B no-tool	GRPO	30	5	6/6
Qwen2.5-7B no-tool	TreeMPO	50	5	6/6
Qwen2.5-7B no-tool	TreeMPO semi-online	7	5	6/6
Qwen2.5-7B + tool	Base	0	5	6/6
Qwen2.5-7B + tool	GRPO	30	5	6/6
Qwen2.5-7B + tool	TreeMPO	50	5	6/6
Qwen3-4B no-tool	Base	0	5	6/6
Qwen3-4B no-tool	GRPO	50	5	6/6
Qwen3-4B no-tool	TreeMPO	30	5	6/6
Qwen3-4B + tool	Base	0	5	6/6
Qwen3-4B + tool	GRPO	40	5	6/6
Qwen3-4B + tool	TreeMPO	50	5	6/6
Gemma-3-4B no-tool	Base	0	5	6/6
Gemma-3-4B no-tool	GRPO	50	5	6/6
Gemma-3-4B no-tool	TreeMPO	40	5	6/6
Mistral-7B no-tool	Base	0	5	6/6
Mistral-7B no-tool	GRPO	40	5	6/6
Mistral-7B no-tool	TreeMPO	20	5	6/6
Llama-3.1 no-tool	Base	0	5	6/6
Llama-3.1 no-tool	GRPO	30	5	6/6
Llama-3.1 no-tool	TreeMPO	40	5	6/6

Table 7: Checkpoint-selection and evaluation-coverage table for the held-out tests in Table 1. All listed rows use per-trial, per-dataset results under the unified judging and scoring pipeline.

E CHECKPOINT AND MECHANISM TABLES

Table 7 records which checkpoints and evaluations support the held-out test table in the main text. Table 8 provides the forced-token mechanism comparison summarized in the main results.

F BRANCHING-BUDGET SENSITIVITY

Our experimental plan includes 8-, 16-, and 32-trajectory variants for the Qwen2.5-7B no-tool setting. The held-out tests do not show a simple monotonic scaling law. Branch budget 32 has a higher held-out test macro average than branch budget 8, but the gap is small and remains within the scale of other run-to-run differences.

Condition	Variant	Best val. epoch	Best val.	Test epoch	Test macro
Qwen2.5-7B no-tool	Non-forced token	20	0.6472 $\pm$ 0.0018	20	0.6294 $\pm$ 0.0085
Qwen2.5-7B no-tool	Forced token	50	0.6523$\pm$0.0016	50	0.6304$\pm$0.0081
Qwen2.5-7B + tool	Non-forced token	70	0.6193 $\pm$ 0.0047	70	0.5962$\pm$0.0044
Qwen2.5-7B + tool	Forced token	50	0.6236$\pm$0.0033	50	0.5836 $\pm$ 0.0116

Table 8: Forced-token mechanism ablation for TreeMPO. Forced-token branching improves the selected validation checkpoint in both Qwen2.5-7B settings, but the held-out test difference is small and reverses in the tool setting.

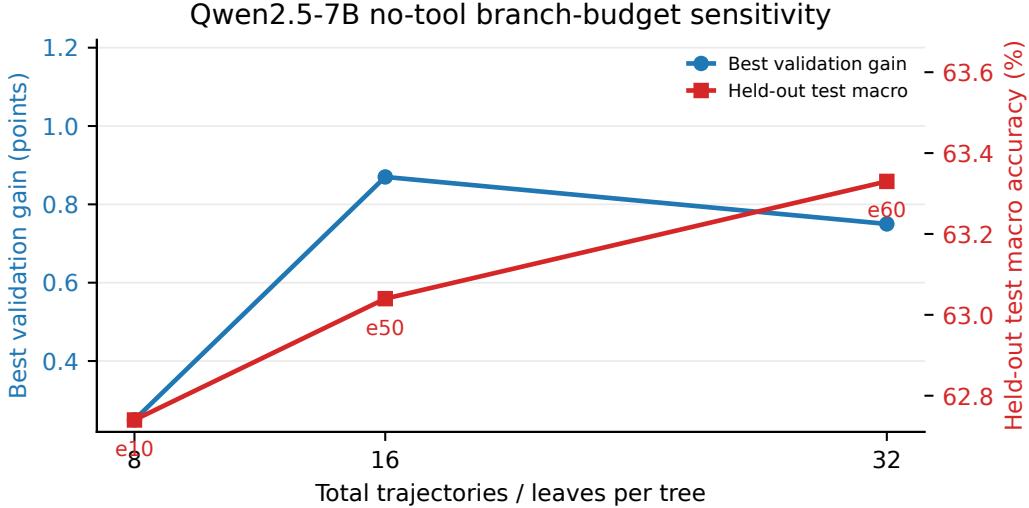


Figure 4: Branching-budget sensitivity for Qwen2.5-7B no-tool TreeMPO variants. The validation line reports the best validation improvement over each run’s epoch-0 checkpoint; the held-out test line reports the five-trial, six-dataset macro accuracy at the validation-selected checkpoint.

G REPORTED RESULTS FROM CLOSELY RELATED SYSTEMS

Table 9 summarizes representative improvements reported by closely related fine-grained, tree-based, or group-relative RL systems. We remove implementation-specific caveats from the table itself and state the key qualification here: these results are not directly comparable to our split-pipeline LoRA experiments because they differ in computation budget, training scale, online versus split rollout policy, majority-vote or pass@1 evaluation, datasets, and whether reference trajectories, process signals, or specialized rollout systems are used.

H IMPLEMENTATION-ALIGNED PSEUDOCODE

Tables 10 and 11 summarize the two implementation paths most relevant for reimplementing. The pseudocode follows the split pipeline used in the experiments: tree rollout produces unjudged trees, a separate judging pass attaches leaf correctness, and training-set generation computes advantages only after judging.

I QUALITATIVE CASE STUDY DETAILS

We inspected generated training trajectories for a Qwen2.5-7B tool-enabled TreeMPO run and selected a NuminaMath question about prices of pencils and notebooks. The correct answer is \$6.75. This example is intentionally a tool-protocol failure rather than a clean success: it shows that segmented credit assignment can expose where a trajectory becomes confused, but also that reference-

Method	Main mechanism	Reported improvement in its paper
SPO / SPO-tree (Guo et al., 2025)	Flexible segment partition, MC segment values, tree-based segment reuse, probability-mask optimization	SPO-chain reports 6–12 point gains over PPO/GRPO on GSM8K; SPO-tree reports 7–11 point gains over GRPO on MATH500 under 2K/4K context evaluation.
TreeRPO (Yang et al., 2025b)	Tree sampling with step/group relative reward expectations	Reports Qwen2.5-Math average Pass@1 increasing from 19.0% to 35.5% , plus 2.9 points over GRPO and 18.1% shorter responses.
TreePO (Li et al., 2025)	Segment-synchronous tree rollout with dynamic branching/fallback and subgroup-relative advantages	Reports GRPO with TreePO sampling improving overall major@16 accuracy from 46.63% to 54.61% ; TreePO advantage adds 2.27–3.6 points while reducing sampling GPU hours by 12–43% .
TEMPO (Tran et al., 2025)	Prefix-to-tree conversion of naturally sampled responses with branch-gated TD corrections	Reports higher validation and final performance than PPO/GRPO on Qwen3-1.7B/4B across math and medical QA, with roughly matched wall-clock time.
TreeAdv (Cao et al., 2026)	Entropy-driven rollout forests with token-level redistribution of leaf advantages	Reports consistent gains over GRPO/GSPO across 10 math benchmarks while using substantially fewer generated tokens under matched supervision/data/decoding budgets.
TreeRL (Hou et al., 2025)	On-policy tree search with value backup from subtree outcomes	Reports improved trajectory diversity and optimization signals relative to independent multi-chain sampling.
TRACE (Zou et al., 2026)	Rollout-budget allocation across prompt roots and intermediate prefixes predicted to yield mixed terminal rewards	Reports Qwen3-14B Multi-Hop QA average accuracy improving by 2.8 points over competitive baselines at equal sampling cost.
APPO (Wang et al., 2026)	Fine-grained tool-calling branching score plus procedure-level advantage scaling	Reports nearly 4-point improvement over strong long-horizon RL baselines on 13 benchmarks with efficient tool calls.
GRPO- λ (Parthasarathi et al., 2025)	Critic-free eligibility-style credit assignment over sequence log-probabilities	Reports 30–40% improved training performance curves and more than 3 points average gain over GRPO, including a 4.5-point gain for a 7B model.
OAR (Li et al., 2026b)	Outcome-sensitivity reshaping by perturbation or gradient proxy	Reports significant gains over strong GRPO baselines, with gradient-based OAR-G approaching perturbation-based OAR-P at low overhead.
LamPO (Yuan et al., 2026)	Pairwise decomposed group advantages with confidence-aware weighting	Reports consistent gains over GRPO and recent RLVR variants on AIME, MATH-500, and GPQA-Diamond.

Table 9: Representative reported improvements from closely related fine-grained, tree-based, or group-relative RL methods. Bold text highlights the numerical gains reported by the corresponding papers. The values are not directly comparable to our results because computation budget, training scale, evaluation policy, datasets, and supervision assumptions differ.

policy-free methods depend on the base model being capable of stopping at tool-call boundaries and waiting for tool responses.

Figure 6 gives the exact prompt seen by the model in the tool TreeMPO setting, including the Qwen chat-template markers. Figures 5 and 7 show exact raw trajectory excerpts from TreeMPO and available ablations. For the ablation panels, we intentionally allow different questions because questions whose sampled leaves are uniformly correct or uniformly incorrect produce zero or near-zero advantages under several credit rules; such cases are uninformative for visualizing how each method distributes credit. The displayed ablation questions are listed in each panel.

The colors are used only to visualize token-level training advantages for readers and automated reviewers; they are not part of the model input or output. Prompt tokens and Python tool outputs have ignored labels and are not colored. Each trajectory uses an adaptive muted red–yellow–green interpolation: the largest absolute advantage in that trajectory saturates to dark red or dark green, so color shows within-trajectory credit structure while the printed numbers show cross-trajectory scale. In the tool-enabled TreeMPO trajectory, the model first sets up the correct equations, then emits a model-generated “executing” block rather than receiving an injected tool response. The later Python call fails because the expected variable is absent from the execution state; the final positive segment manually recomputes the cost and reaches the correct boxed answer.

This appendix is qualitative rather than a substitute for aggregate validation. It illustrates why the TreeMPO design combines guided branching with posterior segment credit. GRPO supplies only a whole-response terminal signal, so it cannot distinguish setup, computation, and repair inside a response. TreeRPO- and TreeRL-style backups can create useful sign changes, but their values are coarser because they are backed up from subtree correctness rates rather than inferred as latent segment contributions. TEMPO-style branching and TreeRL-style entropy branching can expose alternative paths, but their branch construction is less tightly coupled to the token-level intervention used by TreeMPO. In the tool-enabled TreeMPO trajectory, the method preserves the mistaken

Input: question q , rollout config (T, L) , decoding temperature, force-selected-token flag.
Output: rollout tree with prompt/root segment, generated reasoning/tool segments, leaf answers, and no correctness labels.

1. Initialize a tree with the prompt segment for q as the root.
2. While fewer than T trunk leaves exist, generate a complete response from the root prompt and attach the response as a new trunk leaf.
3. While fewer than L total leaves exist:
 - (a) Enumerate existing reasoning tokens. Prompt tokens and tool-response tokens are not candidate branch points.
 - (b) For the first reasoning token of a segment, treat the candidate as node expansion. Exclude first tokens already used by sibling branches. Choose the highest-probability remaining top- k token if its probability relative to the original top token is at least 0.1.
 - (c) For an internal reasoning token, treat the candidate as a segment split. Exclude the token that was actually generated at that position. Choose the highest-probability remaining top- k token if its relative probability is at least 0.1.
 - (d) Score each node-expansion candidate by alternative-token relative probability times a branching-factor penalty. Score each segment-split candidate by alternative-token relative probability times a segment-length penalty. Entropy replaces relative probability only in the TreeRL-style branching ablation.
 - (e) Select the positive-score candidate with maximum score. If no such candidate exists, stop expanding the tree.
 - (f) If the candidate is a segment split, split the selected segment before the selected token; if it is node expansion, branch from the selected node.
 - (g) If forced branching is enabled, append the selected alternative token to the generation prompt, generate the remaining continuation, and then record that selected token and its stored log probability as the first token of the new branch.
 - (h) Attach the generated continuation and update the tree status.
4. Return the tree artifact. Leaf correctness is intentionally absent and is added only by the judging pass.

Table 10: Guided token-level tree construction. This matches the current split-pipeline implementation: branch selection uses model-side top- k token statistics and structural penalties, not posterior uncertainty from leaf judgments.

Question. A local store sells 9 pencils and 5 notebooks for \$3.45. It also sells 6 pencils and 4 notebooks for \$2.40. How much would 18 pencils and 9 notebooks cost?

Colored raw trajectory excerpt. To solve this problem, we need to set up a system of linear equations. Let (p) be the price of one pencil and (n) be the price of one notebook. From the problem statement, $(9p+5n=3.45)$ and $(6p+4n=2.40)$. We can use NumPy's `linalg.solve` function. `python import numpy as np A = np.array([[9, 5], [6, 4]]) b = np.array([3.45, 2.40]) solution = np.linalg.solve(A, b) solution` `python executing ...` `''' The solution to the system is (p approx 0.300) and (n approx 0.315). Now calculate (18p+9n). '''python cost_18_pencils_9_notebooks = 18 * solution[0] + 9 * solution[1] cost_18_pencils_9_notebooks` `'''` `'''output NameError: name 'solution' is not defined '''` It seems there was an issue with referencing the solution. Let me recompute the cost directly using the solved values. `python p = 0.300 n = 0.315 cost_18_pencils_9_notebooks = 18 * p + 9 * n cost_18_pencils_9_notebooks` `'''` `'''output Done '''` The cost of 18 pencils and 9 notebooks is approximately \$6.75. Therefore, the final answer is $(\boxed{6.75})$.

Observed segment advantages. 0.000, -0.870, +0.434.

Figure 5: Qwen2.5-7B tool-calling-compatible failure case. The trajectory reaches the correct final answer, but the missed tool handoff and later state error show why reference-policy-free methods are sensitive to base-model tool-boundary behavior.

pre-tool context, observes the tool response, and assigns positive credit to the later repair segment without treating the entire response as having a single uniform signal.

Input: judged tree with segment set $\mathcal{S}$, leaf labels $y_\ell \in \{+1, -1\}$, and leaf-to-root segment paths.

Output: token-level training trajectories with old log probabilities and clipped advantages.

1. Discard the root prompt segment from posterior variables. For every non-root segment i , initialize mean $m_i = 0$ and log standard deviation $u_i = 0$.
2. For every judged leaf ℓ , build its path indicator $x_{\ell,i}$ over non-root segments.
3. Minimize the negative log posterior

$$\sum_i \frac{(m_i - 0)^2}{2\sigma_m^2} + \sum_i \frac{u_i^2}{2\sigma_u^2} - \sum_\ell \log \Phi \left(\frac{y_\ell \sum_i x_{\ell,i} m_i}{\sqrt{\sum_i x_{\ell,i} \exp(2u_i) + \varepsilon}} \right),$$

using gradient descent with backtracking line search for at most 120 iterations.

4. Clamp every u_i to $[-4, 2]$ after each candidate update.
5. Convert posteriors to raw segment scores by $m_i / \exp(u_i)$, divide by a clamped reasoning-token segment length, then normalize scores across segments by subtracting the segment-score mean and dividing by the segment-score standard deviation. If mean subtraction would flip a segment's sign, set that normalized segment score to zero.
6. Keep only mixed-correctness questions for training-set generation. All-correct and all-incorrect trees provide no contrastive segment-credit signal and are filtered out.
7. Divide each segment score by the number of leaf trajectories that contain that segment so shared-prefix credit is not duplicated across leaves.
8. For each selected leaf trajectory, assign the segment score to generated reasoning/tool-call tokens, assign zero and ignored labels to prompt/tool-response tokens, and retain old token log probabilities from rollout.
9. Balance positive and negative advantage mass across the emitted training set unless a positive-only ablation is explicitly requested, then clip final token advantages to the configured range.

Table 11: MAP segment-credit optimization and conversion to token-level training data. This table describes the default TreeMPO advantage path; TreeRPO, TreeRL, and GRPO ablations replace only the segment-advantage calculation step.

Tool prompt, exact decoded trajectory prefix.

```
<|im_start|>system
You are Qwen, created by Alibaba Cloud. You are a helpful assistant.<|im_end|>
<|im_start|>user
You are a helpful agent that solves the following problem.
Question: A local store sells 9 pencils and 5 notebooks for $3.45$. It also sells 6
pencils and 4 notebooks for $2.40$. How much would 18 pencils and 9 notebooks cost?
A) $6.25$
B) $6.50$
C) $6.75$
D) $7.00$
You are encouraged to use the Python code executor to help you do calculations to ensure
correctness. You can invoke python code by emitting a Python markdown code block like
this:
```python
import numpy as np
A = np.array([[2, 1], [1, -1]])
b = np.array([5, 1])
solution = np.linalg.solve(A, b)
solution
```
You should reason step by step, and put the final answer in a \boxed{ }.
Begin your reasoning:<|im_end|>
<|im_start|>assistant
```

Figure 6: Exact prompt for the Qwen2.5-7B tool-compatible case-study question. This is the prompt used for the displayed trajectory; it is not manually rewritten.

TreeMPO, no-tool. *Question:* A woman is 42 years old and her daughter is 8 years old. In how many years will the mother be three times as old as her daughter?
The sampled no-tool tree solves the age equation directly, so the posterior has little corrective contrast to exploit.
Let's denote the number of years from now when the mother will be three times as old as her daughter as x . Currently, the mother's age is 42 years and the daughter's age is 8 years. In x years, the mother's age will be $42+x$ and the daughter's age will be $8+x$. We can set up the equation $42+x=3(8+x)$, solve it, and obtain $x=9$. The final answer is $\boxed{9}$.
Advantages: +0.074, 0.

GRPO, no-tool. *Question:* Find the auxiliary equation for the ordinary differential equation with constant coefficients $(x^2 D^2 + xD + 1)y = \sin(2 \log x) \sin(\log x)$.
GRPO assigns one terminal group-normalized advantage to the entire response.
To find the auxiliary equation for the given ordinary differential equation with constant coefficients, we first recognize that the equation is not in a standard constant-coefficient form because the operator contains $(x^2 D^2 + xD + 1)$. We rewrite it through the substitution $(t = \log x)$, where (xD) acts like differentiation with respect to (t) . This yields the auxiliary equation $(m^2 + 1 = 0)$.
Advantages: +1.732 for this representative response.

TEMPO-style branching. *Question:* Find the auxiliary equation for the ordinary differential equation with constant coefficients $(x^2 D^2 + xD + 1)y = \sin(2 \log x) \sin(\log x)$.
Spontaneous prefix-tree branching produces a nonzero but fragmented early credit pattern.
To find the auxiliary equation for the given ordinary differential equation, we need to follow these steps. First identify the homogeneous part of the ODE: $((x^2 D^2 + xD + 1)y = 0)$. Assume $(y = x^m)$, substitute $(y' = mx^{m-1})$ and $(y'' = m(m-1)x^{m-2})$, and simplify to $(m^2 + 1 = 0)$.
Advantages: +0.162, +0.111, -0.263, +0.217, +0.633.

TreeRPO-style credit. *Question:* Suppose $g(x) = 5x - 3$. What is $g^{-1}(g^{-1}(14))$?
The child-group win-rate rule gives a strong sign change around the transition from generic inverse derivation to the nested evaluation.
To solve for $(g^{-1}(g^{-1}(14)))$, first find the inverse of $(g(x) = 5x - 3)$. Set $(y = 5x - 3)$, solve $(x = (y + 3)/5)$, and obtain $(g^{-1}(x) = (x + 3)/5)$. Then compute $(g^{-1}(14) = 17/5)$, followed by $(g^{-1}(17/5) = ((17/5) + 3)/5 = 32/25)$. Therefore, the final answer is $\boxed{32/25}$.
Advantages: -0.525, +2.000.

TreeRL-style credit. *Question:* Given $f(x, y) = e^{-(x+y)}$ for $x > 0, y > 0$, compute $P(Y > X + 1)$.
The local-global value rule creates a coarse negative-to-positive split over the integral setup and evaluation.
To compute $(P(Y > X + 1))$, integrate $(e^{-(x+y)})$ over $(x > 0)$ and $(y > x + 1)$. Set up $(\int_0^\infty \int_{x+1}^\infty e^{-(x+y)} dy dx)$. The inner integral becomes $(\int_{2x+1}^\infty e^{-u} du = e^{-(2x+1)})$, after which the remaining integral can be evaluated.
Advantages: -0.219, +0.281.

TreeRL-style branching. *Question:* Evaluate $\lim_{x \rightarrow 0} (1/\tan^2 x - 1/x^2)$.
Entropy-guided branching yields a long recovery pattern with a negative initial transformation and a positive Taylor-expansion segment.
Start by writing $(\tan x = \sin x / \cos x)$, so $(1/\tan^2 x = \cos^2 x / \sin^2 x)$. Combining terms gives $((x^2 \cos^2 x - \sin^2 x) / (x^2 \sin^2 x))$. Then analyze the numerator and denominator as $(x \rightarrow 0)$ using Taylor series for $(\sin x)$ and $(\cos x)$, which reveals the limiting constant after cancellation.
Advantages: -0.323, +0.805.

Figure 7: Representative trajectory excerpts for available Qwen2.5-7B no-tool ablations. Unlike the main tool/no-tool comparison, ablation panels may use different questions so that the visualized examples avoid the uniform-correctness regime where several advantage rules collapse to zero. The examples are reconstructed from judged trajectories for qualitative analysis.